FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Jose Gabriel Valencia
Section:	An-43
Professor:	Ms. Tan
Date:	12/10/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

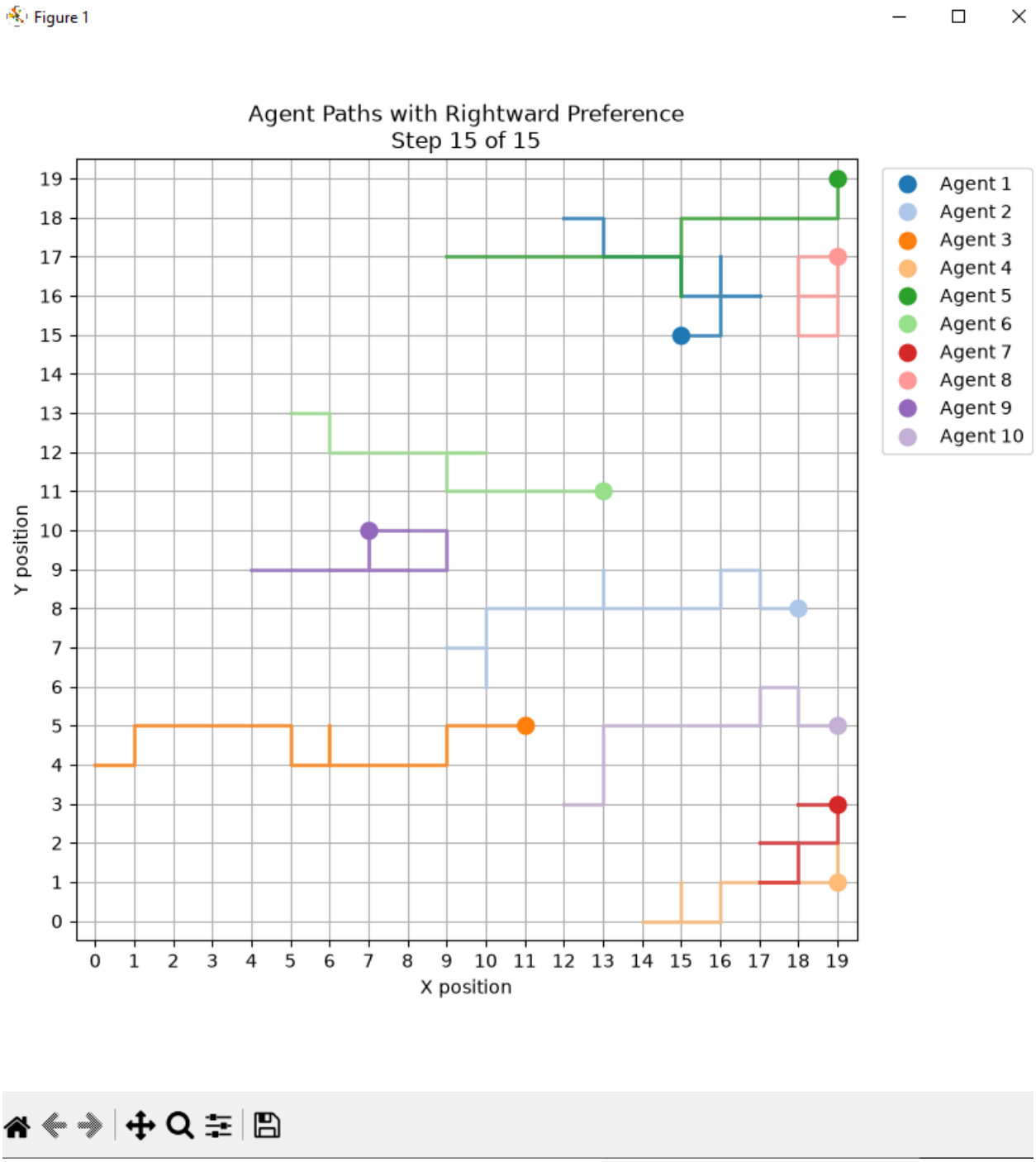
Step 4: Experiment with Parameters

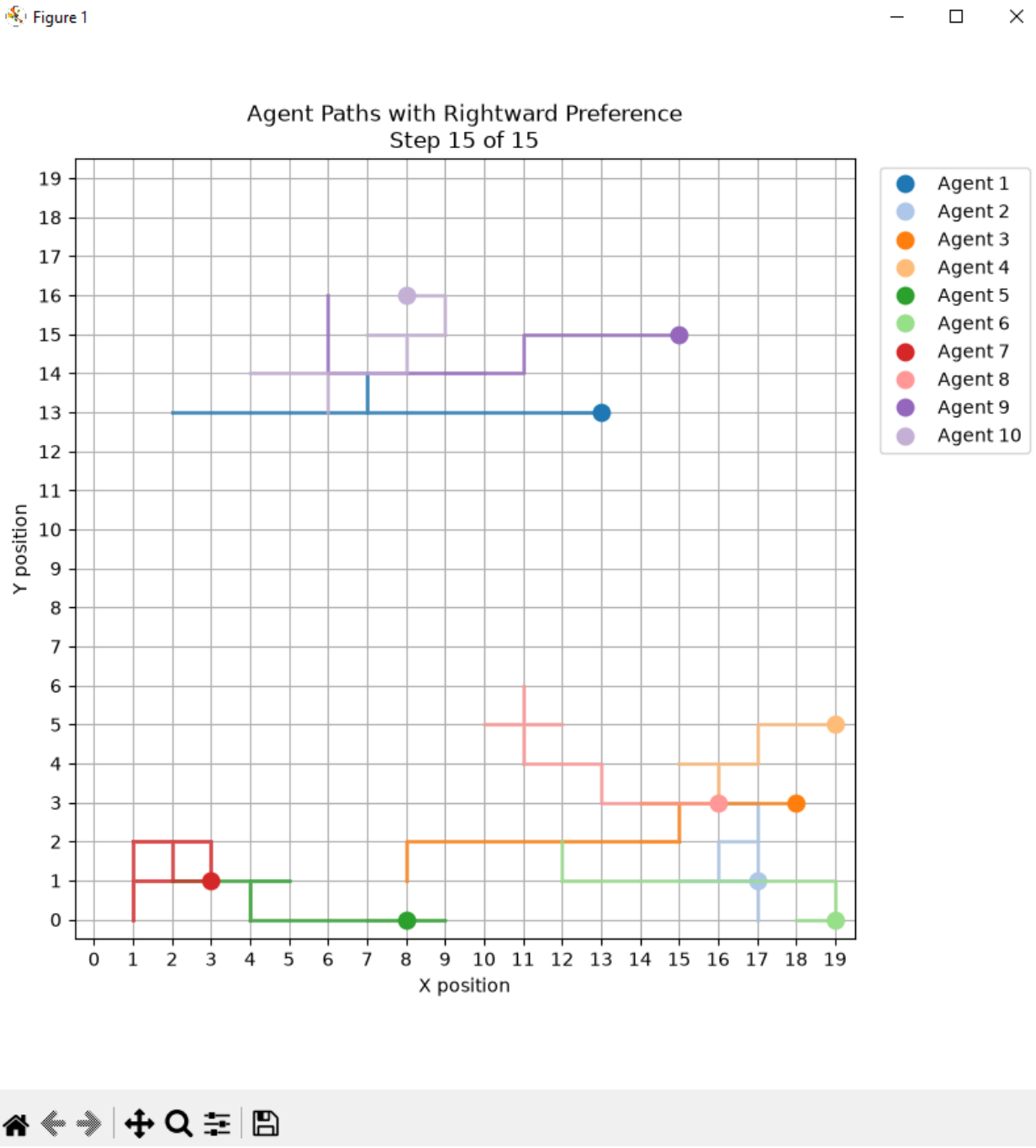
- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.


Figure 1
Agent Paths with Rightward Preference
Step 15 of 15
Y position
X position
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Agent 6
Agent 7
Agent 8
Agent 9
Agent 10



Modified (Updated Code - Laboratory Tasks)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
```

```python
import matplotlib.animation as animation


# ------------------------------------------------
# Built-in movement settings
# ------------------------------------------------

# Preferred direction: right
PREFERRED_DIRECTION = (1, 0)

# Higher value creates a stronger preference.
PREFERENCE_STRENGTH = 3

# Prevent agents from occupying the same grid cell.
AVOID_OTHER_AGENTS = True


# ------------------------------------------------
# Get user input
# ------------------------------------------------

num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

if num_agents > grid_size ** 2:
    raise ValueError(
        "The number of agents cannot exceed "
        "the number of cells in the grid."
    )


# ------------------------------------------------
# Agent definition
# ------------------------------------------------

class RandomWalker(ap.Agent):

    def setup(self):
        # Store every position visited by this agent.
        self.path = []

    def choose_direction(self):
```

```
        directions = [
            (1, 0),   # right
            (-1, 0),  # left
            (0, 1),   # up
            (0, -1)   # down
        ]

        # Add the preferred direction several times.
        # This makes movement to the right more likely.
        movement_choices = (
            directions
            + [PREFERRED_DIRECTION] * PREFERENCE_STRENGTH
        )

        return random.choice(movement_choices)

    def get_next_position(self):
        dx, dy = self.choose_direction()
        x, y = self.position

        new_x = x + dx
        new_y = y + dy

        # Keep the agent inside the grid.
        new_x = max(
            0,
            min(self.model.p.grid_size - 1, new_x)
        )

        new_y = max(
            0,
            min(self.model.p.grid_size - 1, new_y)
        )

        return new_x, new_y


# ------------------------------------------------
# Model definition
# ------------------------------------------------

class RandomWalkModel(ap.Model):
```

```
    def setup(self):
        # Create the agents.
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            RandomWalker
        )

        # Create a list containing every grid cell.
        available_cells = [
            (x, y)
            for x in range(self.p.grid_size)
            for y in range(self.p.grid_size)
        ]

        # Give each agent a different starting position.
        starting_positions = random.sample(
            available_cells,
            self.p.agents
        )

        for agent, position in zip(
            self.agents,
            starting_positions
        ):
            agent.position = position
            agent.path.append(position)

    def step(self):
        # Record all currently occupied positions.
        occupied_positions = {
            agent.position for agent in self.agents
        }

        # Randomize which agent moves first.
        movement_order = list(self.agents)
        random.shuffle(movement_order)

        for agent in movement_order:
            current_position = agent.position
            next_position = agent.get_next_position()

            # Temporarily remove the agent's current cell.
```

```
            occupied_positions.remove(current_position)

            if (
                not AVOID_OTHER_AGENTS
                or next_position not in occupied_positions
            ):
                # Move if the destination is unoccupied.
                agent.position = next_position
            else:
                # Remain in place if another agent is there.
                agent.position = current_position

            occupied_positions.add(agent.position)

            # Save the latest position in the agent's path.
            agent.path.append(agent.position)


# ------------------------------------------------
# Create the model
# ------------------------------------------------

parameters = {
    "agents": num_agents,
    "grid_size": grid_size
}

model = RandomWalkModel(parameters)
model.setup()


# ------------------------------------------------
# Create the graph
# ------------------------------------------------

fig, ax = plt.subplots(figsize=(8, 8))

ax.set_xlim(-0.5, grid_size - 0.5)
ax.set_ylim(-0.5, grid_size - 0.5)

ax.set_xticks(range(grid_size))
ax.set_yticks(range(grid_size))
```

```
ax.grid(True)
ax.set_aspect("equal")

ax.set_xlabel("X position")
ax.set_ylabel("Y position")
ax.set_title("Agent Paths with Rightward Preference")

colors = plt.cm.tab20.colors

# Create one dot and one path line for every agent.
agent_dots = []
agent_lines = []

for index, agent in enumerate(model.agents):
    color = colors[index % len(colors)]

    dot, = ax.plot(
        agent.position[0],
        agent.position[1],
        marker="o",
        markersize=9,
        color=color,
        linestyle="None",
        label=f"Agent {index + 1}"
    )

    path_line, = ax.plot(
        [],
        [],
        color=color,
        linewidth=2,
        alpha=0.75
    )

    agent_dots.append(dot)
    agent_lines.append(path_line)

# Display a legend when the number of agents is manageable.
if num_agents <= 10:
    ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1))


# ------------------------------------------------
```

```
# Animation update
# ------------------------------------------------

def update(frame):
    model.step()

    for index, agent in enumerate(model.agents):
        # Update the agent's current position.
        agent_dots[index].set_data(
            [agent.position[0]],
            [agent.position[1]]
        )

        # Get the complete recorded path.
        x_path = [
            position[0] for position in agent.path
        ]

        y_path = [
            position[1] for position in agent.path
        ]

        # Display the recorded path.
        agent_lines[index].set_data(x_path, y_path)

    ax.set_title(
        "Agent Paths with Rightward Preference\n"
        f"Step {frame + 1} of {num_steps}"
    )

    return agent_dots + agent_lines


simulation_animation = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps,
    interval=500,
    blit=False,
    repeat=False
)

plt.tight_layout()
```

```
plt.show()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?

2. How does increasing the number of steps affect the **spread of agents**?

3. Do the agents ever move outside the grid? Why or why not?

4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. If there are higher number of agents that the grid size agent traffic happens where sometimes if a agent gets stuck in between multiple agents it causes grid locks where in the agents will not be able to move due to the movement rule of avoiding collision.

2. Given with our current movement rule if you increase the number of steps they generally create a much more larger area in their starting position but some of the agents just revisit their previous path creating a loop this is due to the random movements of the agent.

3. No because the code limits us to the X and Y coordinate of maximum if there is a an attempt to move towards the edge it would move to another grid since its already at maximum grid.

4. The simulation can be used in a real-life phenomena in the creation of pathways for pedestrian people. Creating solution for crowded area and avoidance of directions that consist collision. Though it might be un avoidable but it is neglectable and can be reduce for people to not create collision.